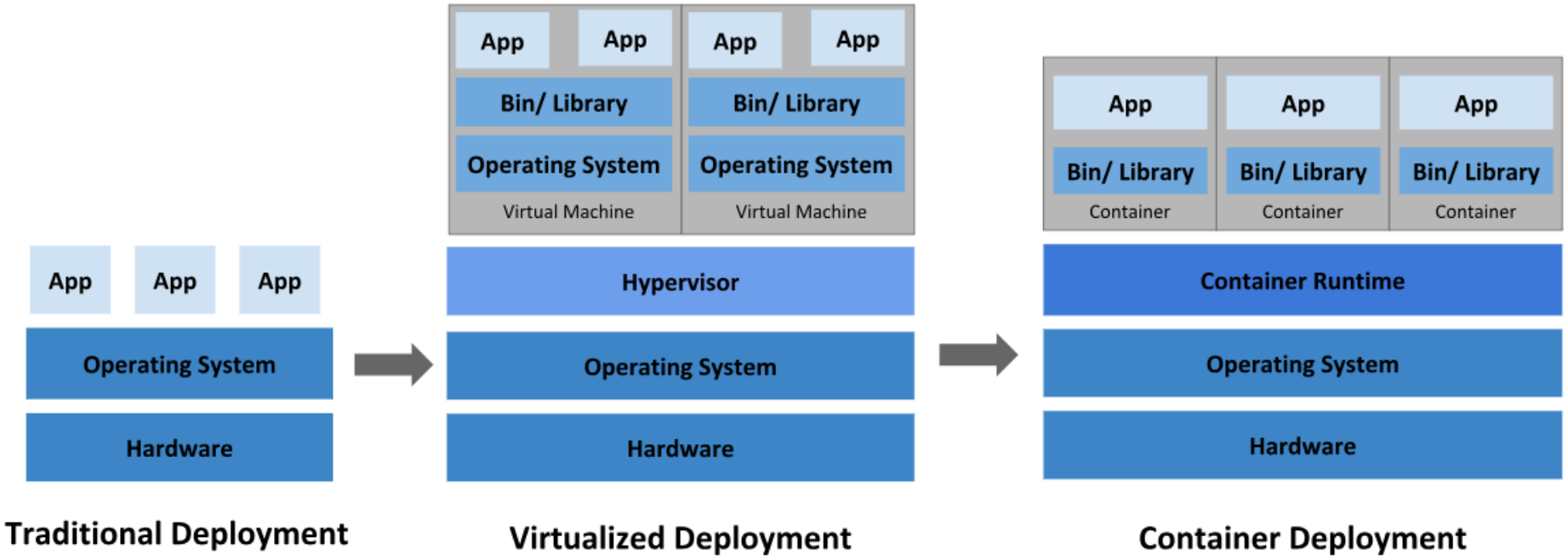
Kubernetes

- Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.
- The name Kubernetes originates from Greek, meaning helmsman or pilot. K8s as an abbreviation results from counting the eight letters between the "K" and the "s

- Google open-sourced the Kubernetes project in 2014. Kubernetes combines over 15 years of Google's experience running production workloads at scale with best-of-breed ideas and practices from the community


App
App
App
Operating System
Hardware
Traditional Deployment
App
App
App
App
Bin/ Library
Bin/ Library
Operating System
Operating System
Virtual Machine
Virtual Machine
Hypervisor
Operating System
Hardware
Virtualized Deployment
App
App
App
Bin/ Library
Bin/ Library
Bin/ Library
Container
Container
Container
Container Runtime
Operating System
Hardware
Container Deployment

- Virtualization allows you to run multiple Virtual Machines (VMs) on a single physical server's CPU. Virtualization allows applications to be isolated between VMs and provides a level of security as the information of one application cannot be freely accessed by another application.

- utilization of resources
- Migration
- Interoperability

Containers are similar to VMs, but they have relaxed isolation properties to share the Operating System (OS) among the applications. Therefore, containers are considered lightweight

a container has a dedicated filesystem, share of CPU, memory, process space, and more.

- Agile application creation and deployment: increased ease and efficiency of container image creation compared to VM image use.
- Continuous development, integration, and deployment: provides for reliable and frequent container image build and deployment with quick and efficient rollbacks (due to image immutability).
- Dev and Ops separation of concerns: create application container images at build/release time rather than deployment time, thereby decoupling applications from infrastructure.
- Observability: not only surfaces OS-level information and metrics, but also application health and other signals.
- Environmental consistency across development, testing, and production: runs the same on a laptop as it does in the cloud.

- Cloud and OS distribution portability: runs on Ubuntu, RHEL, CoreOS, on-premises, on major public clouds, and anywhere else.
- Application-centric management: raises the level of abstraction from running an OS on virtual hardware to running an application on an OS using logical resources.
- Loosely coupled, distributed, elastic, liberated micro-services: applications are broken into smaller, independent pieces and can be deployed and managed dynamically – not a monolithic stack running on one big single-purpose machine.
- Resource isolation: predictable application performance.
- Resource utilization: high efficiency and density

- **Service discovery and load balancing** Kubernetes can expose a container using the DNS name or using their own IP address. If traffic to a container is high, Kubernetes is able to load balance and distribute the network traffic so that the deployment is stable.
- **Storage orchestration** Kubernetes allows you to automatically mount a storage system of your choice, such as local storages, public cloud providers, and more.
- **Automated rollouts and rollbacks** You can describe the desired state for your deployed containers using Kubernetes, and it can change the actual state to the desired state at a controlled rate. For example, you can automate Kubernetes to create new containers for your deployment, remove existing containers and adopt all their resources to the new container.
- **Automatic bin packing** You provide Kubernetes with a cluster of nodes that it can use to run containerized tasks. You tell Kubernetes how much CPU and memory (RAM) each container needs. Kubernetes can fit containers onto your nodes to make the best use of your resources.

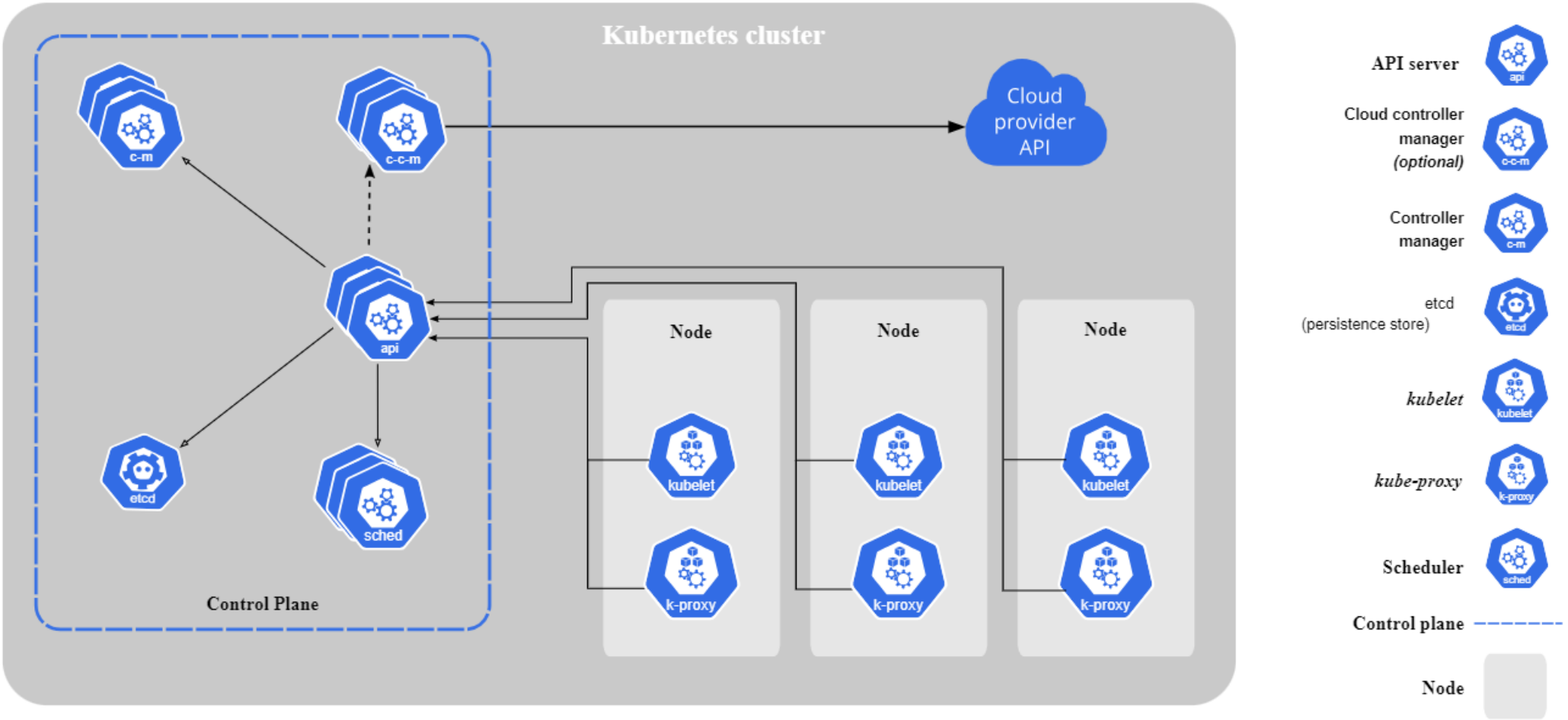
- **Self-healing** Kubernetes restarts containers that fail, replaces containers, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.
- **Secret and configuration management** Kubernetes lets you store and manage sensitive information, such as passwords, OAuth tokens, and SSH keys. You can deploy and update secrets and application configuration without rebuilding your container images, and without exposing secrets in your stack configuration.
- **Batch execution** In addition to services, Kubernetes can manage your batch and CI workloads, replacing containers that fail, if desired.
- **Horizontal scaling** Scale your application up and down with a simple command, with a UI, or automatically based on CPU usage.
- **IPv4/IPv6 dual-stack** Allocation of IPv4 and IPv6 addresses to Pods and Services
- **Designed for extensibility** Add features to your Kubernetes cluster without changing upstream source code

Kubernetes:

- Does not limit the types of applications supported. Kubernetes aims to support an extremely diverse variety of workloads, including stateless, stateful, and data-processing workloads. If an application can run in a container, it should run great on Kubernetes.
- Does not deploy source code and does not build your application. Continuous Integration, Delivery, and Deployment (CI/CD) workflows are determined by organization cultures and preferences as well as technical requirements.
- Does not provide application-level services, such as middleware (for example, message buses), data-processing frameworks (for example, Spark), databases (for example, MySQL), caches, nor cluster storage systems (for example, Ceph) as built-in services. Such components can run on Kubernetes, and/or can be accessed by applications running on Kubernetes through portable mechanisms, such as the Open Service Broker.

- Does not dictate logging, monitoring, or alerting solutions. It provides some integrations as proof of concept, and mechanisms to collect and export metrics.
- Does not provide nor mandate a configuration language/system (for example, Jsonnet). It provides a declarative API that may be targeted by arbitrary forms of declarative specifications.
- Does not provide nor adopt any comprehensive machine configuration, maintenance, management, or self-healing systems.
- Additionally, Kubernetes is not a mere orchestration system. In fact, it eliminates the need for orchestration. The technical definition of orchestration is execution of a defined workflow: first do A, then B, then C. In contrast, Kubernetes comprises a set of independent, composable control processes that continuously drive the current state towards the provided desired state. It shouldn't matter how you get from A to C. Centralized control is also not required. This results in a system that is easier to use and more powerful, robust, resilient, and extensible.

- A Kubernetes cluster consists of a set of worker machines, called nodes, that run containerized applications. Every cluster has at least one worker node.
- The worker node(s) host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.
- This document outlines the various components you need to have for a complete and working Kubernetes cluster.


Kubernetes cluster
c-m
c-c-m
Cloud provider API
api
etcd
sched
Control Plane
Node
kubelet
k-proxy
Node
kubelet
k-proxy
Node
kubelet
k-proxy
API server
Cloud controller manager (optional)
Controller manager
etcd (persistence store)
kubelet
kube-proxy
Scheduler
Control plane
Node

Control Plane Components

kube-apiserver

- The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.
- The main implementation of a Kubernetes API server is kube-apiserver. kube-apiserver is designed to scale horizontally—that is, it scales by deploying more instances. You can run several instances of kube-apiserver and balance traffic between those instance

etcd

- Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.
- If your Kubernetes cluster uses etcd as its backing store, make sure you have a back up plan for the data.
- You can find in-depth information about etcd in the official documentation.

kube-scheduler

- Control plane component that watches for newly created Pods with no assigned node, and selects a node for them to run on.
- Factors taken into account for scheduling decisions include: individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and deadlines

kube-controller-manager

- Control plane component that runs controller processes.
- Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process.
- There are many different types of controllers. Some examples of them are:
- Node controller: Responsible for noticing and responding when nodes go down.
- Job controller: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
- EndpointSlice controller: Populates EndpointSlice objects (to provide a link between Services and Pods).
- ServiceAccount controller: Create default ServiceAccounts for new namespaces.

cloud-controller-manager

- A Kubernetes control plane component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.
  - The cloud-controller-manager only runs controllers that are specific to your cloud provider. If you are running Kubernetes on your own premises, or in a learning environment inside your own PC, the cluster does not have a cloud controller manager.
- As with the kube-controller-manager, the cloud-controller-manager combines several logically independent control loops into a single binary that you run as a single process. You can scale horizontally (run more than one copy) to improve performance or to help tolerate failures.

- The following controllers can have cloud provider dependencies:
- Node controller: For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
- Route controller: For setting up routes in the underlying cloud infrastructure
- Service controller: For creating, updating and deleting cloud provider load balancers

Node Components

- Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment

- kubelet
- An agent that runs on each node in the cluster. It makes sure that containers are running in a Pod.
- The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. The kubelet doesn't manage containers which were not created by Kubernetes

A way to expose an application running on a set of Pods as a network service.

A node is a worker machine in Kubernetes.

A lightweight and portable executable image that contains software and all of its dependencies.

A Pod represents a set of running containers in your cluster.

kube-proxy

kube-proxy is a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept.

kube-proxy maintains network rules on nodes. These network rules allow network communication to your Pods from network sessions inside or outside of your cluster.

kube-proxy uses the operating system packet filtering layer if there is one and it's available. Otherwise, kube-proxy forwards the traffic itself.

Container runtime

A fundamental component that empowers Kubernetes to run containers effectively. It is responsible for managing the execution and lifecycle of containers within the Kubernetes environment.

Kubernetes supports container runtimes such as containerd, CRI-O, and any other implementation of the Kubernetes CRI (Container Runtime Interface).

Ensures a copy of a Pod is running across a set of nodes in a cluster.

Manages a replicated application on your cluster.

Addons

Addons use Kubernetes resources (DaemonSet, Deployment, etc) to implement cluster features. Because these are providing cluster-level features, namespaced resources for addons belong within the `kube-system` namespace.

Selected addons are described below; for an extended list of available addons, please see Addons.

https://kubernetes.io/docs/concepts/cluster-administration/addons/

DNS

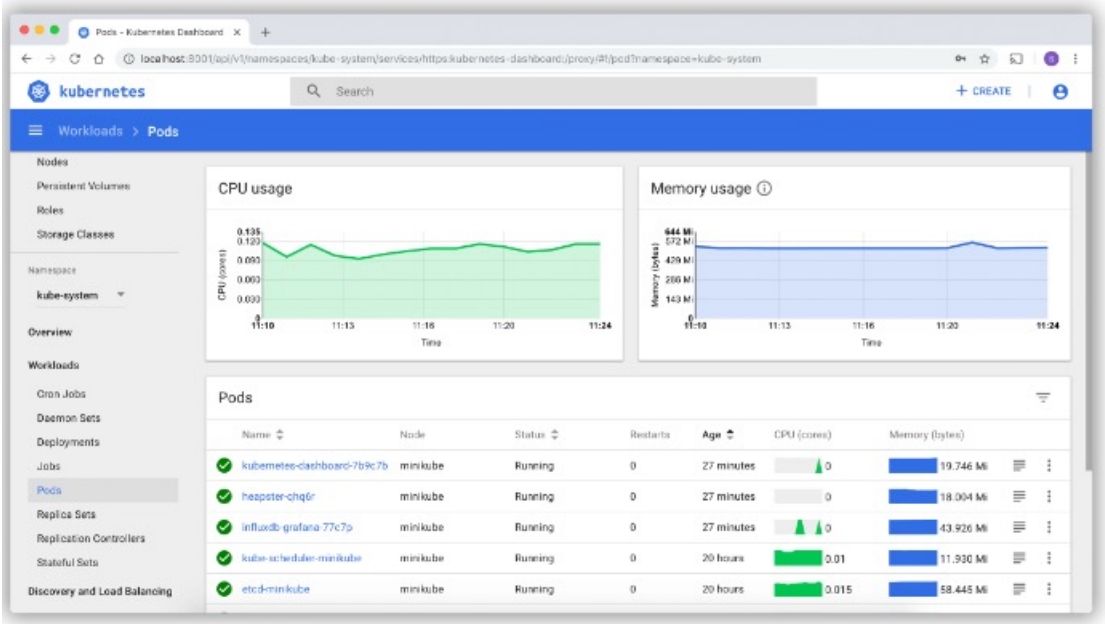
While the other addons are not strictly required, all Kubernetes clusters should have cluster DNS, as many examples rely on it.

Cluster DNS is a DNS server, in addition to the other DNS server(s) in your environment, which serves DNS records for Kubernetes services.

Containers started by Kubernetes automatically include this DNS server in their DNS searches.

Web UI (Dashboard)

Dashboard is a general purpose, web-based UI for Kubernetes clusters. It allows users to manage and troubleshoot applications running in the cluster, as well as the cluster itself.



Container Resource Monitoring

Container Resource Monitoring records generic time-series metrics about containers in a central database, and provides a UI for browsing that data.

Cluster-level Logging

A cluster-level logging mechanism is responsible for saving container logs to a central log store with search/browsing interface.

Network Plugins

Network plugins are software components that implement the container network interface (CNI) specification. They are responsible for allocating IP addresses to pods and enabling them to communicate with each other within the cluster.

Data Containerization

Docker Architecture: https://docs.docker.com/get-started/overview/ (https://docs.docker.com/get-started/overview/)

Go Programming: https://go.dev/ (https://go.dev/)

Docker Containerization

Building Blocks of the Docker Stack:

- **Docker CLI.**
- **Docker Engine.**
- **Docker Compose:**
  - Compose has commands for managing the whole lifecycle of your application:
    - Start, stop, and rebuild services
    - View the status of running services
    - Stream the log output of running services
    - Run a one-off command on a service

Documentations: https://docs.docker.com/desktop/) (https://docs.docker.com/desktop/)) :

- **Docker Engine (https://docs.docker.com/engine/)**
- Docker CLI client
- **Docker Buildx (https://docs.docker.com/build/)**
- **Extensions (https://docs.docker.com/desktop/extensions/)**
- **Docker Compose (https://docs.docker.com/compose/)**
- **Docker Content Trust (https://docs.docker.com/engine/security/trust/)**
- **Kubernetes (https://github.com/kubernetes/kubernetes/)**
- **Credential Helper (https://github.com/docker/docker-credential-helpers/)**

Docker Capabilities

1. **File Composition/Formatting**
2. **Command-Line Interfacing**
3. **Application Programming Interfaces (APIs)**
   1. **The Docker Engine API**
      Docker Engine is an open source containerization technology for building and containerizing your applications. Docker Engine acts as a client-server application with:
      - A server with a long-running daemon process
      - APIs which specify interfaces that programs can use to talk to and instruct the Docker

daemon.
- A command line interface (CLI) client

2. **The Docker Registry HTTP API => Interactions with Instances of the Docker Registry**
   Docker Registry is a stateless, highly scalable server-side application that stores and lets you distribute Docker images.
   - tightly control where your images are being stored
   - fully own your images distribution pipeline
   - integrate image storage and distribution tightly into your in-house development workflow
3. **The Docker Hub API => Docker Hub Interactions**
   Docker Hub (https://hub.docker.com/) is a service provided by Docker for finding and sharing container images with your team. It's the world's largest repository of container images with an array of content sources including container community developers, open source projects and independent software vendors (ISV) building and distributing their code in containers.
4. **The DVP Data API => Verified Publisher Analytics Interactions**

4. **Drivers and Specifications**
   1. **Image Specification**
   2. **Registry Token Authentication**
   3. **Registry Storage Drivers**

Docker System Operations:

- Show the version of the Docker stack: **docker version**
- Show information about the stack including the server: **docker info**
- Login to a Docker registry: **docker login**
- Log out: **docker logout**
- Show low-level information about an object: **docker inspect** the_object
- Remove unused images, containers, and networks: **docker system prune**

Docker Networks Operations:

- View available networks: d**ocker network ls**
- Remove a network: **docker network rm** network_name
- Show information about a network: **docker network inspect** network_name
- Connect a container to a network: **docker network connect** network_name container_name
- Disconnect a container from a network: **docker network disconnect** networ_name container_name

Docker Plugin Operations:

- Enable a Docker plugin: **docker plugin enable [plugin]**
- Disable a Docker plugin: **docker plugin disable [plugin]**
- Create a plugin from config.json and rootfs: **docker plugin create [plugin] [path-to-data]**
- View details about a file system: **docker plugin inspect [plugin]**

- Remove a plugin: **docker plugin rm [plugin]**

Components of the Docker Desktop (A GUI for managing images, containers and applications).

Three Views

The **Images** view

displays a list of your Docker images

- allows you to run an image as a container,
- pull the latest version of an image from Docker Hub, and
- inspect images.
- it displays a summary of the vulnerability scanning report using Snyk. In addition, the **Images** view contains clean-up options to remove unwanted images from the disk to reclaim space. If you are logged in, you can also see the images you and your organization have shared on Docker Hub. For more information, see **Explore your images (https://docs.docker.com/desktop/use-desktop/images/)** .
- Image Operations:

Image Operations (docker ==>)

- Build Image: **docker build** docker_file_path
- Build image at current path: **docker build .**
- Build an image from a file: **docker build -f** file_name
- Pull an image from a repository: **docker pull** image_name
- Push an image to a registry: **docker push** image_name
- Create an image from a tarball: **docker import** URL/File_name
- Create an image from a container: **docker commit** container_name image_name
- Tag an image: **docker tag** image_name image_name:tag
- List all locally stored top-level images: d**ocker images**
- Show the history of an image: **docker history** image_name
- Remove an image: **docker rmi** image_name
- Load an image from a docker tar archive file: **docker load --image** tar_file_name
- Save an image to a docker tar archive file: **docker save** image_name **>** tar_file_name
- **Search docker hub for images: docker search** query
- Remove all unused images/images that do not have any containers associated with them : **docker image prune -a**
- Sort Images
  - In use
  - Unused
  - Dangling

The **Containers** view

provides a runtime view of all your containers and applications.

- It allows you to interact with containers and applications, and manage the lifecycle of your applications directly from your machine.
- This view also provides an intuitive interface to perform common actions to inspect, interact with, and manage your Docker objects including containers and Docker Compose-based applications. For more information, see **Explore running containers and applications (https://docs.docker.com/desktop/use-desktop/container/)** .

- ## Container Operations
  - List all running containers: **docker ps -a**
  - List the running and stopped all containers: **docker ps -s**
  - Create a container: **docker create image_name**
  - create a container with a pseudo tty: **docker create -it** image_name
  - Rename **docker rename** container_name new_name
  - List port numbers
  - Force remove a running container: **docker rm** -f container_nme
  - Remove a stopped container: **container rm** container_name
  - Pause/Resume
  - Stop/Start
  - Delete
  - Open the application in VS code
  - Open the port exposed by the container in a browser
  - Copy docker run. This allows you to easily share container run details or modify certain parameters

The **Volumes** view

- displays a list of volumes and allows you to easily create and delete volumes and see which ones are being used. For more information, see **Explore volumes (https://docs.docker.com/desktop/use-desktop/volumes/)** .

- ## Volume Operations
  - Search
  - Sort
    - Name
    - Dated Created
    - Size
    - Remove

```
C:\Users\khali>docker search helloworld
NAME                                        DESCRIPTION                                     STARS     OFFICIAL   AUTOMATED
supermanito/helloworld                      学习资料                                          270
karthequian/helloworld                      A simple helloworld nginx container to get y…   18                   [OK]
strm/helloworld-http                        A hello world container for testing http bal…   6                    [OK]
buoyantio/helloworld                                                                        4
helloworldcontainers/java-spring-boot-hello-world                                           4
ibmcom/helloworld                           A sample used by IBM Cloud Code Engine          2
wouterm/helloworld                           A simple Docker image with an Nginx server …   1                    [OK]
hakanssonlasse/helloworld                                                                   1
testcontainers/helloworld                   A Docker image to support Testcontainers' se…   1
helloworld2121/ashish                       hi                                              1
helloworld9x/kubia                                                                          0
hivetechnologies/helloworldarm              Helloworld Webserver written in GoLang for t…   0
cstxmohammad/helloworld                     to run the image use following command:  doc…   0
apache/causeway-app-helloworld              Apache Causeway App Helloworld                  0
agrawals18/helloworld                                                                       0
neyahpeterson/helloworldui                                                                  0
nimak/helloworld                                                                            0
helloworld102444/studentserver                                                              0
containerinstance/helloworld                Demo image for SSL connection inside Node.js    0
isaactl/helloworld                                                                          0
moseers/helloworld                                                                          0
neyahpeterson/helloworldapi                                                                 0
mperel/helloworld                                                                           0
simongdavies/helloworld-aci-cnab                                                            0
helloworld2121/walmart21                                                                    0
```

Docker Image/Service Composition Example:

Step 1: Define the application dependencies

a. app.py

b. Requirements.txt

Step 2: Create a Dockerfile

app.py | docker-compose.yml | Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.7-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run"]
```

- Build an image starting with the Python 3.7 image.
- Set the working directory to `/code`.
- Set environment variables used by the `flask` command.
- Install gcc and other dependencies

- Copy `requirements.txt` and install the Python dependencies.
- Add metadata to the image to describe that the container is listening on port 5000
- Copy the current directory `.` in the project to the workdir `.` in the image.
- Set the default command for the container to `flask run`.

Step 3: Define services in a Compose file

app.py docker-compose.yml Dockerfile

```
version: "3.9"
services:
  web:
    build: .
    ports:
      - "20000:5000"
  redis:
    image: "redis:alpine"
```

This Compose file defines two services: `web` and `redis`.

The `web` service uses an image that's built from the `Dockerfile` in the current directory. It then binds the container and the host machine to the exposed port, `8000`. This example service uses the default port for the Flask web server, `5000`.

The `redis` service uses a public **Redis** (https://registry.hub.docker.com/_/redis/) image pulled from the Docker Hub registry.

Step 4: Build and run your app with Compose

`docker compose up`

Step 5 Push the image to the Docker Hub

`a. Create a web version of the image and tag it`

`docker tag` `image_id` `your_hub_name/hub_repository_name:tag`

Example: docker tag a5e0a24a719 abuosbakh/cs490:ver1.0

`b.List images`

`docker images`

`c. Push image to the Docker Hub`

`docker push your_hub_name/hub_repository_name:tag`

Example: docker push abuosbakh/cs490:ver1.0

d. pull the following image to your Docker Desktop:

docker pull abuosbakh/cs490:ver1.0

Docker System Operations:

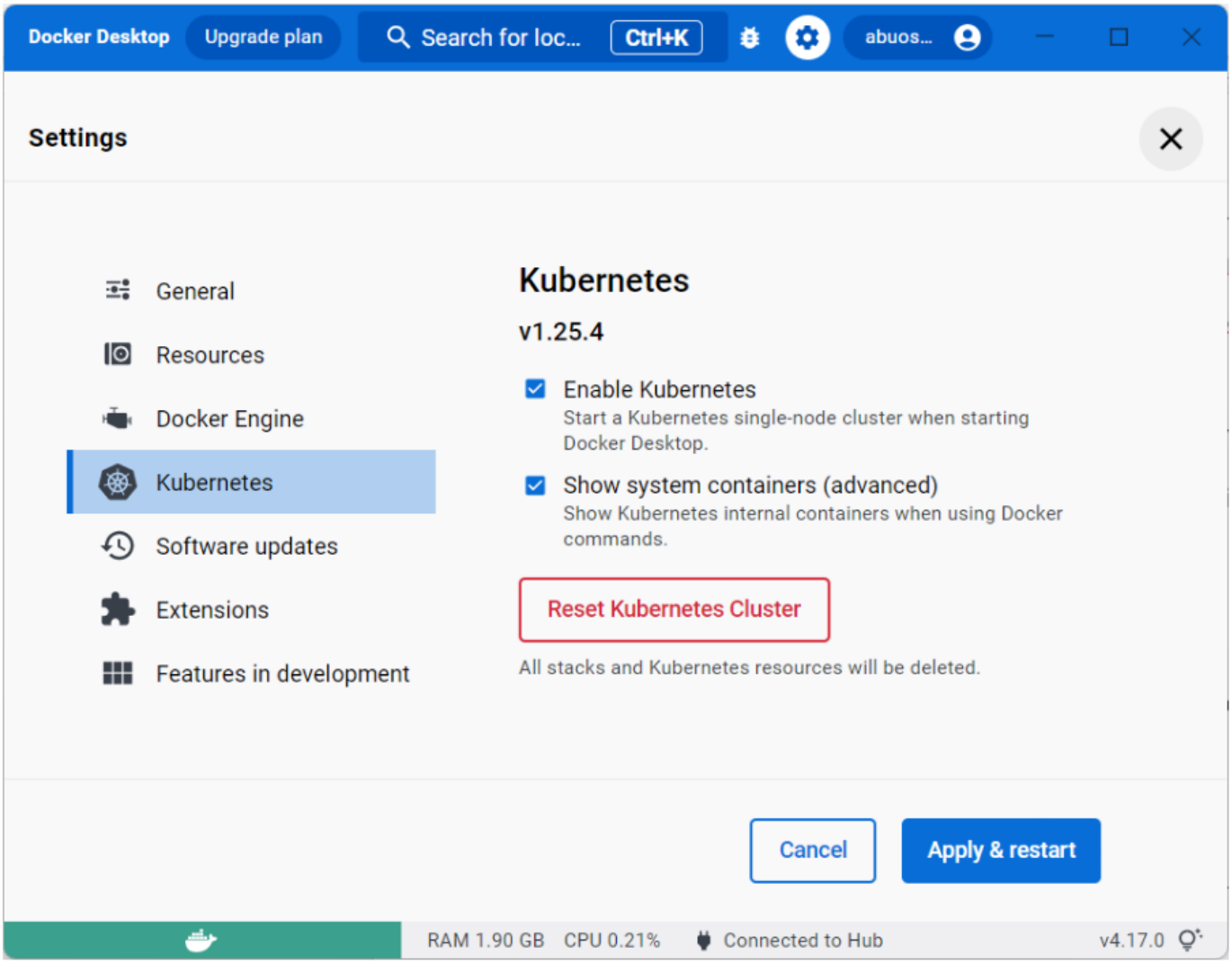
- Show the version of the Docker stack: **docker version**
- Show information about the stack including the server: **docker info**
- Login to a Docker registry: **docker login**
- Log out: **docker logout**
- Show low-level information about an object: **docker inspect** the_object
- Remove unused images, containers, and networks: **docker system prune**

Docker Networks Operations:

- View available networks: d**ocker network ls**
- Remove a network: **docker network rm** network_name
- Show information about a network: **docker network inspect** network_name
- Connect a container to a network: **docker network connect** network_name container_name
- Disconnect a container from a network: **docker network disconnect** networ_name container_name

Docker Plugin Operations:

- Enable a Docker plugin: **docker plugin enable [plugin]**
- Disable a Docker plugin: **docker plugin disable [plugin]**
- Create a plugin from config.json and rootfs: **docker plugin create [plugin] [path-to-data]**
- View details about a file system: **docker plugin inspect [plugin]**
- Remove a plugin: **docker plugin rm [plugin]**



Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.

It groups containers that make up an application into logical units for easy management and discovery. Kubernetes builds upon 15 years of experience of running production workloads at Google, combined with best-of-breed ideas and practices from the community.

Automated rollouts and rollbacks

Kubernetes progressively rolls out changes to your application or its configuration, while monitoring application health to ensure it doesn't kill all your instances at the same time. If something goes wrong, Kubernetes will rollback the change for you. Take advantage of a growing ecosystem of deployment solutions.

Service discovery and load balancing

No need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

Storage orchestration

Automatically mount the storage system of your choice, whether from local storage, a public cloud provider such as AWS or GCP, or a network storage system such as NFS, iSCSI, Ceph, Cinder.

Self-healing

Restarts containers that fail, replaces and reschedules containers when nodes die, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.

Secret and configuration management

Deploy and update secrets and application configuration without rebuilding your image and without exposing secrets in your stack configuration.

Automatic bin packing

Automatically places containers based on their resource requirements and other constraints, while not sacrificing availability. Mix critical and best-effort workloads in order to drive up utilization and save even more resources.